

Assignment: AI-Powered Market Intelligence & Trend Analytics Platform

Objective

Build a production-grade backend system that ingests real-world data from a third-party API, processes it, stores it in MongoDB, performs analytics, and exposes insights through a FastAPI-based REST API. The system must be deployed and publicly accessible.

Learning Outcomes

By completing this assignment, you will:

- Design and build a real-world backend system
 - Integrate external APIs into your application
 - Work with unstructured and semi-structured data in MongoDB
 - Perform data analysis and trend detection
 - Build scalable APIs using FastAPI
 - Deploy a production-ready backend system
-

System Overview

Pipeline: Third-Party API → Data Ingestion → MongoDB → Data Processing → FastAPI → Deployment

Third-Party API (Choose ONE)

You must integrate ONE of the following:

- News API (Recommended)
 - Alpha Vantage (Stocks/Crypto)
 - Adzuna API (Job Market)
-

Functional Requirements

1. Data Ingestion

- Fetch data from the selected API
- Normalize and clean data

- Store raw data in MongoDB (`raw_data` collection)

2. Data Processing

- Transform raw data into structured format
- Extract:
 - Keywords
 - Sentiment (basic rule-based or API-based)
 - Trend score
- Store processed data in `processed_data` collection

3. Database Design (MongoDB)

Raw Collection Example:

```
{
  "title": "...",
  "source": "...",
  "published_at": "...",
  "content": "...",
  "category": "business"
}
```

Processed Collection Example:

```
{
  "title": "...",
  "sentiment": "positive/negative",
  "keywords": ["AI", "economy"],
  "score": 0.87,
  "trend_score": 12
}
```



Data Analysis Requirements

You must implement:

A. Trend Detection

- Identify top keywords (last 24h / 7 days)

B. Sentiment Analysis

- Classify data into positive, negative, neutral

C. Time-Series Analysis

- Count items per day
 - Identify growth trends
-

FastAPI Requirements

Implement the following endpoints:

- `POST /ingest` → Fetch and store data
- `GET /trends?days=7` → Return trending keywords
- `GET /insights` → Aggregated insights
- `GET /search?q=keyword` → Search functionality
- `GET /analytics/summary` → Summary statistics

Mandatory Features:

- Pydantic models
 - Query parameters
 - Pagination
 - Error handling
-

Advanced Features (Choose ANY TWO)

- Background tasks (FastAPI)
 - Caching (in-memory or Redis)
 - Logging system
 - Rate limiting
 - Data validation pipeline
-

Deployment Requirements

You must deploy your backend using ONE of the following:

- Render
- Railway

Deployment must include:

- Environment variables (.env)
 - Production server (uvicorn/gunicorn)
 - MongoDB Atlas integration
-



Timeline (2 Days)

Day 1:

- Setup project
- API integration
- Store raw data
- Data processing
- MongoDB schema design
- Aggregations

Day 2:

- Build all API endpoints
- Implement analytics
- Add advanced features
- Deploy system
- Testing



Deliverables

You must submit:

1. GitHub Repository
2. Live API URL
3. Postman Collection or Swagger Docs
4. README including:
 5. Project overview
 6. Architecture diagram
 7. API documentation
 8. Sample outputs



Evaluation Criteria

Criteria	Weight
Functionality	25%
Code Quality	20%
API Design	15%

Criteria	Weight
Data Processing	15%
Deployment	15%
Advanced Features	10%

Rules

- No copy-paste projects
- Must use real API data
- Code must be modular and clean
- Proper README is mandatory

Bonus (Optional)

- Add LLM-based summarization
- Build a frontend dashboard
- Add scheduled automation

Final Note

This is not a basic CRUD project. Treat it like a real production system. Focus on scalability, clean architecture, and real-world practices.